Memóriakezelés háttere: egy program alapvetően valamilyen (bináris végrehajtható) fájl formában helyezkedik el a háttértárban. Végrehajtáshoz be kell tölteni a memóriába.
A főmemória és a regiszterek az egyetlen olyan tárolók, amikhez a CPU közvetlenül hozzáfér. A regisztereket 1 CPU ciklus (vagy kevesebb) alatt el lehet érni, míg a memória eléréséhez több ciklus kell. A cache a fő memória és a CPU regiszterek között helyezkednek el. A memória védelme szükséges, hogy biztosítsuk a helyes műveleteket.
A programkódhoz és a program változókhoz valamikor memória címeket kell rendelni (address binding). (Ez történhet a betöltés előtt, közben, vagy akár utána is. Megkülönböztetünk *fordítási , betöltési (szerkesztési) és futási időben történő tárfoglalás*t (címkapcsolást).
A processzor kapacitásának jobb kihasználása megköveteli, hogy egyszerre több processz osztozzon a memórián.

Logikai és fizikai címtér:
• Logikai cím = a CPU által generált cím (virtuális cím). A felhasználói programok csak ilyeneket használnak.
• Fizikai cím = reális cím.
Fordítási és betöltési időben csak a logikai címtér érhető el.
MMU (memóriamenedzsment-egység): ez végzi a logikai címek fizikai címekké való leképezését. *Legegyszerűbb megoldás*: a program által történő címzésnél a kért címhez hozzáadódik a bázisregiszter (itt relokációs reg.) értéke.
Overlay: A felhasználói program logikájába (szubrutin struktúrájába) beépített dinamika.
Alapötlete: a teljes programnak (kód és adat) csak az a része legyen benn az operatív memóriában, amelyre ténylegesen szükség van. (dinamikus betöltés) Egy szubrutin nem töltődik be, amíg meg nem hívódik. Minden szubrutin a lemezen található, áthelyezhető bináris formában.

Relokáció: a program (logikai címtartományának) eltolása.
Védelem: a processzusok alaphelyzetben csak a hozzájuk rendelt címtartományt használhatják.
Swapping (csere): egy folyamatot ki lehet swappelni egy időre a memóriából a háttérben lévő tárra és később vissza lehet tölteni, hogy a futását folytassa. A háttértároláshoz a winchester elég nagy ahhoz, hogy egy memória image másolatait minden user számára tárolja.
A folyamatok kivitele és behozatala mindig valamilyen prioritás alapú ütemezési algoritmus szerint történnek. Az alacsony prioritású folyamatokat ki lehet swappelni, míg a magas prioritásúakat be lehet swappelni.
A *dispatcher* dönti el, hogy egy, az ütemező által betöltött folyamat elfér-e a memóriában vagy swapping szükséges.

Dinamikus tár-hozzárendelési probléma: hogyan lehet kielégíteni egy adott méretű allokációs (tárfoglalási) igényt?
• First-fit: foglaljuk le az első lyukat, amely elég hosszú!
• Best-fit: foglaljuk le az elég hosszú lyukak közül azt, amelynek hossza legközelebb esik a szükséges hosszhoz!
• Worst-fit: foglaljuk le a leghosszabb lyukat (ha az elég hosszú)!

Alprogram: főprogramtól (annak változóitól, konkrét értékeitől) függetlenül általánosan megírt programrészlet.
Előnyei:
- több személy írhatja egyszerre a programot
- külön tesztelhető
- a program áttekinthetősége jobb
- külön file-ként elkészíthető, így több programba is beékelhető
- egy programon belül többször meghívható más-más konkrét értékkel
- rekurzió használata

Két csoportba sorolhatjuk: *(sok nyelv (C, C++, Java, C#) nem tesz különbséget köztük)*
- *eljárás*: a program környezetén vagy a változókon végez módosításokat.
- *függvény*: valamilyen értéket számít ki, a változókra és a program környezetére nincs hatással (nincs mellékhatás)
Specifikáció: Az alprogram specifikációja az az interface, amelyet a hívó program ismer.

Visszatérés: A függvényből való visszatérés során meg kell adnunk a visszatérési értéket. A legtöbb nyelvben erre a return utasítás használható, amely után a visszatérési értéket adjuk meg.
Alprogramok hívása: az alprogram nevének megadásával és az aktuális paraméterek megadásával történik.

Paraméterátadás: Az információáramlás irányának szempontjából a paramétereket három csoportra oszthatjuk: Bemenő paraméterek. Kimenő paraméterek. Be- és kimenő paraméterek.
Paraméterátadási módok: A paraméterátadás módszere alapján a következő két fő csoport különböztethető meg:
- Érték szerinti: A bemenő paraméterek átadására használható módszer. A formális paraméter lokális változó, az aktuális paraméter értéke ide másolódik. Az alprogram megváltoztathatja a formális paraméter értékét, de ez nincs hatással az aktuális paraméter értékére.
- Cím szerinti paraméterátadás: Be- és kimenő paraméterek kezelésére használhatjuk. Használatakor az alprogram egy hivatkozást kap az aktuális paraméterhez, így az alprogram és a hívó ugyanazt a memóriaterületet (ugyanazt a változót) érik el. A paraméternek olyan objektumnak kell lennie, amelyre hivatkozás (memóriacím) mutathat.

Hatókör: Egy deklaráció (típus, konstans, változó, eljárás, függvény) érvényességi köre, az a kör, melyben őt ismerik, hivatkozhatnak rá.
Lokális deklaráció: függvény blokkjában levő deklaráció, csak a függvény ismeri.
- Élettartama: Amíg az eljárás fut.
- Keletkezése: Az eljárás hívásakor.
- Megszűnése: Az eljárásból való visszatéréskor.
- Tárolása: A veremszegmensben (stack).

Globális deklaráció: függvényen kívüli deklaráció, más blokk is ismerheti.
- Élettartama: Elérhetők a program futásának teljes időtartama alatt, a program bármely eljárásából.
- Tárolása: Az adatszegmensben.